

SOGR: SEMANTIC OBJECT GRAPH RUNTIME

A Structured Incremental Architecture for Long-Context Language Modeling

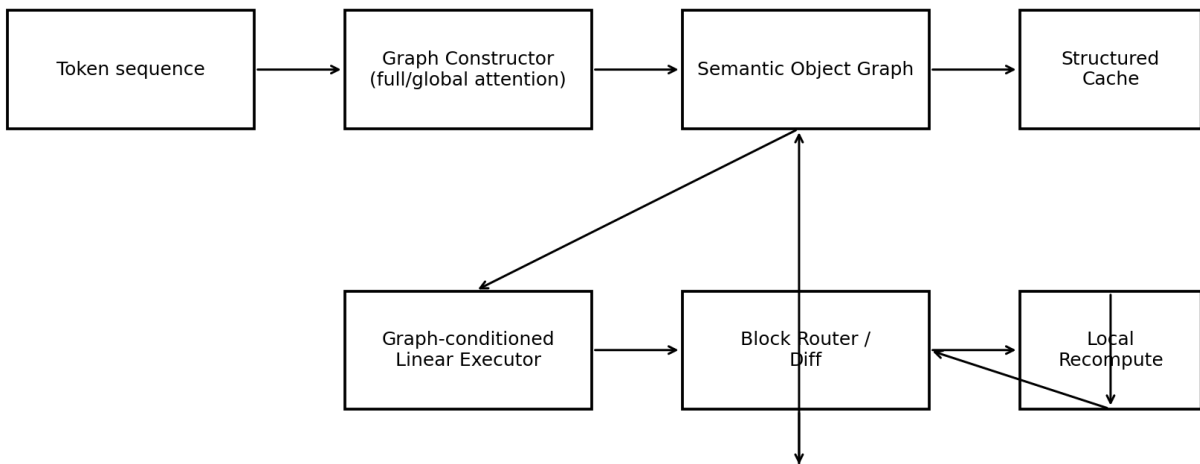
Zeping Tu

Independent Research Draft · Version 0.1 · September 2026

Abstract. This paper proposes SOGR, a language-model architecture that treats semantic dependency structure as an explicit computational intermediate representation rather than forcing every layer to rediscover relationships over a token sequence. The central hypothesis is that a relatively infrequent global-attention stage can construct or repair a Semantic Object Graph (SOG), after which graph-conditioned linear state propagation performs the high-frequency computation. A structured cache stores node, edge, and block states; block-level change detection provides a coarse routing mechanism analogous in spirit to dirty-region propagation in reactive user-interface systems. When a local variable, entity, or relation changes, only its dependency subgraph is recomputed, while unrelated state is retained. The proposed architecture combines four ideas: (1) low-frequency global graph construction, (2) high-frequency graph-conditioned linear execution, (3) periodic global graph repair, and (4) hierarchical block-level skipping. We derive a conditional complexity model showing that the method can outperform dense attention only when graph construction is amortized, dependency density remains sufficiently sparse, and graph-maintenance overhead is bounded. This manuscript is a research proposal, not a report of completed experiments; no empirical performance claim is made. The intended contribution is an explicit architectural hypothesis and a falsifiable experimental program.

Keywords: semantic object graph; incremental computation; structured cache; linear attention; sparse computation; block routing; long-context language modeling

Semantic Object Graph Runtime (SOGR)



Global construction is low-frequency; graph propagation and local recomputation are high-frequency.

Figure 1. Overview of the proposed SOGR runtime. Global attention is used as a low-frequency graph-construction/repair operator; graph-conditioned linear operators execute repeated updates; block routing and structured caching reduce unnecessary recomputation.

1. Introduction

Transformer-based language models represent context primarily as a sequence of token states and repeatedly apply attention to infer interactions among those states. This design is powerful because it makes few assumptions about which relationships matter. Its cost, however, is that relationship discovery is repeatedly entangled with information propagation. Even when the effective dependency pattern is sparse, a conventional dense attention operator does not expose that sparsity as a first-class computational object.

A different design question is therefore proposed here: what if the model first constructed an explicit, reusable representation of language relationships, and subsequent computation operated on that representation rather than rediscovering the relationships at every layer? The proposal is inspired by compiler intermediate representations, abstract syntax trees, dependency graphs, and reactive UI systems such as virtual DOMs. The analogy is not that natural language is a DOM; rather, both systems separate structure from execution and can propagate changes through a dependency graph.

SOGR formalizes this idea as a four-stage runtime. First, a global attention module constructs a Semantic Object Graph (SOG). Second, graph-conditioned linear operators propagate state along the graph. Third, after a configurable number of linear steps, a global attention module repairs or extends the graph. Fourth, hierarchical block summaries decide whether a region is stable enough to skip fine-grained computation. A structured cache stores the resulting state and invalidation metadata.

The proposal is deliberately narrower than a claim that sparse attention or linear attention is sufficient. Existing efficient architectures already demonstrate that linear-time sequence processing and hybrid attention can be practical. The proposed novelty is the organization of computation: relationship discovery becomes an explicit, cacheable state; computation becomes an incremental graph update problem.

2. Design Motivation

The first-generation efficiency idea considered in this research direction was token selection: identify salient positions and discard the rest. Such methods can reduce computation, but they still treat the token sequence as the fundamental object. The proposed architecture instead asks whether the underlying dependency structure can be made explicit before deciding what to compute.

Consider the sentence “Alice bought a laptop in Hangzhou because her old computer failed.” A token-only representation requires the model to infer that “her” refers to Alice, that “old computer” is distinct from “laptop,” and that the causal clause explains the purchase context. A structured representation can expose these as candidate nodes and edges. Importantly, the graph is not required to be linguistically perfect; it may contain weighted, uncertain, or revisable edges.

The key engineering principle is therefore: do expensive global relationship discovery less frequently, and reuse the discovered structure for many cheaper propagation steps. This is analogous to compiling a program into an intermediate representation and then repeatedly executing the compiled structure, rather than reparsing the source code for every operation.

3. Semantic Object Graph

Let a sequence of L token embeddings be $X = \{x_1, \dots, x_L\}$. SOGR introduces a graph $G=(V,E,A)$, where V contains token, phrase, entity, event, or latent semantic objects; E contains typed dependency relations; and A stores relation confidence or routing weights. The graph may be hierarchical: token-level nodes can belong to phrase nodes, which belong to sentence or block nodes.

A minimal graph node can be represented as $N_i = (h_i, \text{type}_i, \text{span}_i, \text{parent}_i, \text{version}_i)$, where h_i is the current hidden state and version_i supports cache invalidation. An edge is $E_{ij} = (r_{ij}, w_{ij}, \text{version}_{ij})$, where r_{ij} is a relation type and w_{ij} is a learned or inferred strength. Unlike a fixed syntactic parse, the graph is allowed to be revised by later global passes.

The important distinction from conventional sparse attention is that the graph is intended to become a persistent computational state. A sparse attention mask is usually a per-layer routing decision. In SOGR, the graph can persist across multiple linear-execution stages and can therefore serve as an index for incremental recomputation.

4. Architecture

The architecture consists of four principal operators: Global Graph Constructor (GGC), Graph-conditioned Linear Executor (GLE), Periodic Graph Repair (PGR), and Hierarchical Block Router (HBR).

4.1 Global Graph Constructor. A full-attention or otherwise high-capacity module examines the current state and proposes graph nodes and weighted relations. The expensive operation is intentionally low-frequency. It is not assumed that one graph remains correct indefinitely.

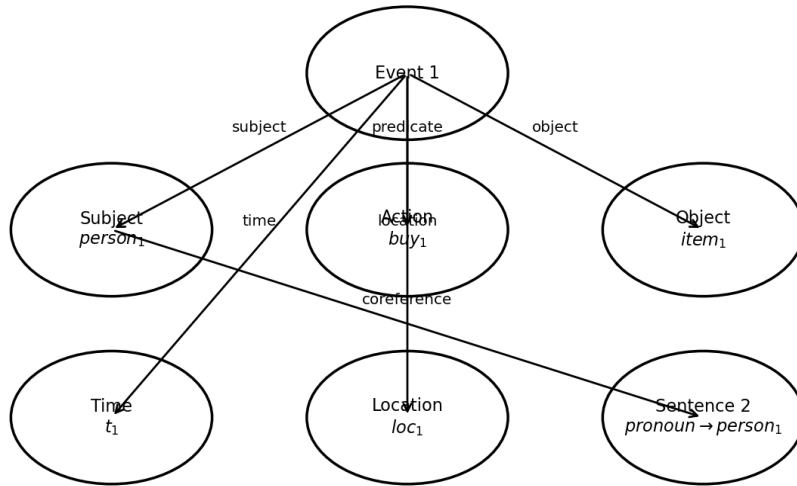
4.2 Graph-conditioned Linear Executor. Given G , a linear operator propagates state only along graph-supported relations. One generic formulation is $h'_i = U h_i + \sum_{j \in N(i)} w_{ij} V_{r_{ij}} h_j$, followed by a gated state update. The implementation can use gated linear attention, delta-rule memory, SSM-like state transitions, or another linear-time mechanism. The architecture therefore does not depend on one particular linear-attention formulation.

4.3 Periodic Graph Repair. After K linear execution steps, a global module re-evaluates the graph. It may add edges, remove stale edges, alter weights, split nodes, merge nodes, or simply confirm that the current structure remains valid. This is the mechanism intended to address the finite-state and retrieval limitations of purely linear models.

4.4 Hierarchical Block Router. Tokens are grouped into blocks. Each block maintains a compact summary z_b and a change indicator Δ_b . If a new query or upstream state has sufficiently low interaction with a stable block, the block can bypass fine-grained computation. The routing unit is intentionally block-sized because contiguous block execution is more hardware-friendly than arbitrary token-level sparsity.

4.5 Structured Cache. The cache stores node states, edge states, block summaries, and dependency versions. A change to node i increments its version and invalidates only descendants or dependents reachable through the graph. Cache validity is therefore structural rather than purely positional.

Example Semantic Dependency Graph



A change to *person*₁ invalidates only dependent nodes/edges rather than the entire sequence.

Figure 2. Example semantic dependency graph. The graph stores typed relations and can propagate invalidation from a changed semantic object.

5. Mathematical Formulation

Let G_t denote the graph at global repair step t . The global constructor is $G_t = C_\theta(X_t, G_{t-1})$, where C_θ may use full attention. Between repairs, K linear updates are applied: $H_{t,k+1} = F_\phi(H_{t,k}, G_t, M_t)$, where M_t denotes cached state.

A generic graph-conditioned linear update can be written as:

$$H' = \sigma(H W_0 + P_G(H) W_1), P_G(H)_i = \sum_{j \in N(i)} A_{ij} H_j.$$

Here P_G is a graph propagation operator. If the graph contains E active edges, the propagation term is $O(Ed)$ for hidden width d , ignoring implementation-specific projection costs. The central requirement is that E be substantially smaller than L^2 and that graph maintenance not erase the savings.

For block routing, let b index blocks and z_b be a block summary. Define a stability score $s_b = g(z_b, q)$, where q is the current query/state. A block is skipped when $s_b < \tau$ and its dependency version has not changed. The active computation becomes a function of the active block set B_{active} rather than all B blocks.

The cache invalidation rule can be expressed as $I_{t+1} = \text{Desc}_G(\Delta_t) \cup \Delta_t$, where Δ_t is the set of changed nodes and Desc_G denotes their dependent descendants. This gives an explicit target for incremental computation: recompute only I_{t+1} , subject to a safety policy that can force global repair when invalidation grows beyond a threshold.

6. Conditional Complexity Analysis

The proposal does not justify a blanket claim of $O(L)$ complexity. If global graph construction occurs every K layers, the amortized cost contains a global term. Let $c_g L^2$ be the cost of one global pass, $c_e E$ the cost of one graph execution, and $c_m I$ the cost of maintaining/invalidation, where I is the number of affected nodes or edges. Over K execution steps, a simplified average cost is:

$$C_{\text{avg}} \approx (c_g L^2)/K + c_e E + c_m I/K.$$

The architecture is advantageous when this quantity is lower than the corresponding dense-attention baseline while preserving model quality. If $E = \rho L^2$, then the graph term is still quadratic and the architecture may provide little asymptotic benefit. The desired regime is therefore $\rho \ll 1$, large K , and small average invalidation size I .

This condition is important. The proposal should not be presented as “Transformer becomes $O(L)$ ” without qualification. Its intended benefit is amortized and conditional: expensive global relationship discovery is reused, while stable or unrelated regions are not repeatedly recomputed.

A further potential benefit appears during incremental decoding or editing. If only a small semantic region changes, the relevant cost may scale with the affected dependency subgraph rather than with the complete context, provided the graph remains valid and the task permits such locality.

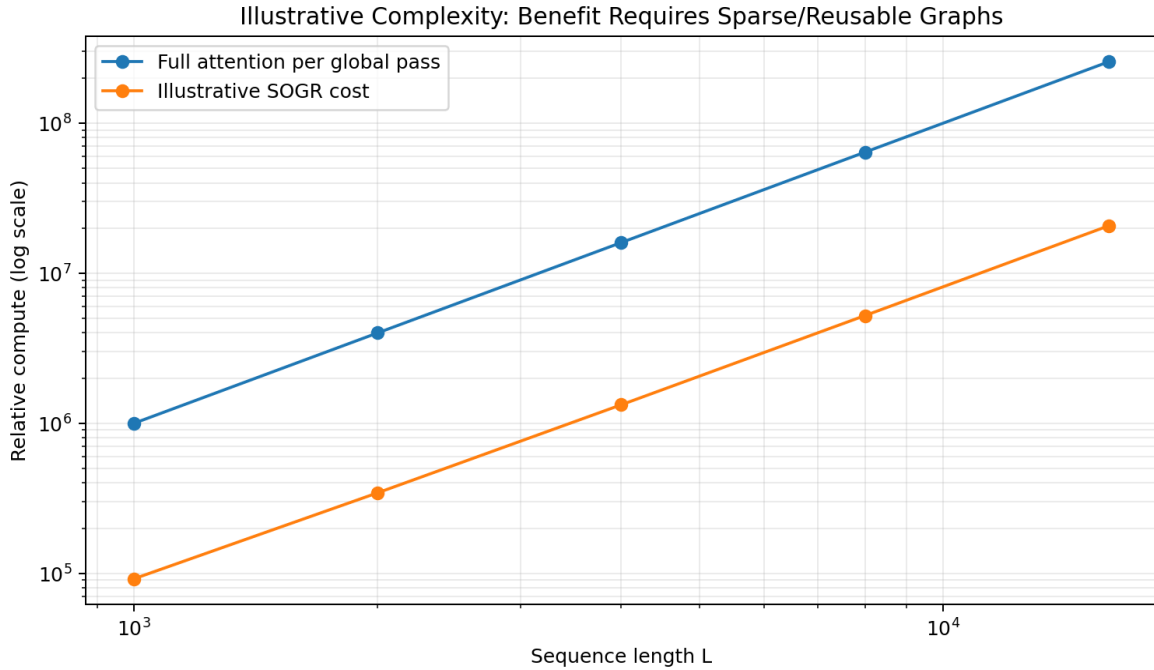


Figure 3. Illustrative comparison only. The proposed curve assumes a sparse graph and amortized global passes; it is not an experimental result.

7. Relationship to Existing Work

SOGR is related to several established directions but is not identical to any single one. Linear attention reformulates attention to obtain linear sequence complexity and exposes a recurrent/fast-weight interpretation. Mamba and related selective state-space models similarly pursue efficient long-sequence processing while addressing content-dependent memory. Kimi Linear demonstrates that hybrid architectures can interleave linear attention with periodic global attention and achieve strong long-context efficiency.

Sparse attention methods reduce computation by restricting the set of interactions, often using fixed windows, learned routing, hashing, or chunk selection. The conceptual distinction proposed here is that the interaction structure is treated as a persistent intermediate representation rather than merely a mask for one attention operation.

The closest conceptual relatives outside LLMs are incremental compilers, databases with materialized views, dependency-based build systems, and reactive UI frameworks. In each case, a dependency structure allows local changes to propagate without recomputing unrelated state. The research question is whether the same computational principle can be made reliable for language representations, where dependencies are uncertain and context-sensitive.

This manuscript therefore claims an architectural hypothesis, not priority over all constituent techniques. A formal novelty/priority determination would require a systematic literature and patent search beyond the scope of this draft.

8. Training Strategy

A practical training strategy is to start from a conventional Transformer or hybrid linear-attention model and introduce auxiliary supervision for graph quality. Candidate graph edges can be derived from attention patterns, coreference links, dependency parses, latent probes, or learned edge predictors. The graph should ultimately be trainable end-to-end rather than requiring manual annotation.

One possible objective is $L = L_{lm} + \lambda_r L_{relation} + \lambda_s L_{stability} + \lambda_i L_{invalidation}$. L_{lm} is the ordinary language-modeling loss. $L_{relation}$ encourages useful dependency prediction. $L_{stability}$ encourages graph persistence when the semantic state is unchanged. $L_{invalidation}$ penalizes unnecessary recomputation or incorrect locality assumptions.

A curriculum can begin with fixed graphs or teacher-generated graphs, then progressively remove external structural supervision. This allows the model to learn the runtime semantics before being asked to discover the structure independently.

9. Experimental Program

The first experiment should not train a billion-parameter model. A small controlled model is sufficient to test the central hypothesis. The key comparison is not only perplexity or benchmark score but compute per useful update and dependency locality.

Recommended evaluations include: (1) language modeling perplexity at equal FLOPs; (2) long-context retrieval such as MQAR and needle-in-a-haystack variants; (3) exact-copy and associative recall tests; (4) semantic-edit tests in which one entity or fact is modified and the required recomputation region is measured; (5) graph precision/recall against automatically generated dependency probes; (6) end-to-end GPU throughput using block-sparse kernels; and (7) ablations over graph refresh frequency K , block size, graph density ρ , and cache invalidation threshold τ .

A decisive experiment is an edit locality benchmark. Construct documents with explicit dependency chains. Change one fact in the middle, then measure how many hidden states must be recomputed to recover the same output quality as a full recomputation. The central hypothesis predicts that the affected subgraph will remain substantially smaller than the complete context for a meaningful class of edits.

The final experiment should compare SOGR against full attention, a strong linear-attention baseline, a standard hybrid architecture, and a chunk-sparse attention baseline under matched parameter count, training tokens, hardware, and quality targets.

10. Limitations and Failure Modes

Natural language dependencies are not deterministic AST edges. Pronouns, discourse relations, negation, irony, world knowledge, and long-range semantic interactions can invalidate an apparently local graph. A safe system therefore needs uncertainty-aware edges and mechanisms for forcing global repair.

Graph construction itself can be expensive. If the constructor must repeatedly perform nearly full attention, the expected savings can disappear. The architecture consequently depends on a sufficiently long interval between graph repairs and a graph that remains useful during that interval.

Hardware sparsity is another limitation. Arbitrary graph traversal can be slower than dense matrix multiplication on GPUs. Block-level grouping is therefore not an optional implementation detail; it is part of the architectural hypothesis. The graph should expose contiguous, batchable regions whenever possible.

Finally, a smaller computation graph does not imply identical model quality. The graph may discard a weak but important long-range dependency. The system must therefore optimize for safe approximation, not merely maximum sparsity.

11. Conclusion

SOGR proposes a shift in the computational abstraction of language models. Instead of treating the token sequence as the only persistent state and rediscovering relationships at every layer, the architecture introduces a Semantic Object Graph as a cacheable intermediate representation. Global attention constructs and repairs that representation; graph-conditioned linear operators perform repeated propagation; structured caches retain stable state; and hierarchical block routing avoids unnecessary computation.

The proposal is intentionally falsifiable. Its success depends on measurable conditions: graph sparsity, graph stability, invalidation locality, constructor frequency, and hardware efficiency. If those conditions fail, the architecture reduces to an expensive hybrid attention system and offers little advantage. If they hold, the same principle could provide a route toward incremental language-model runtimes in which the unit of computation is not a token sequence but a dependency subgraph.

The central research question is therefore not “how can Transformer attention be made slightly cheaper?” but “can language be compiled into a stable computational structure that allows the model to update only what changed?” SOGR is a concrete architectural hypothesis for testing that question.

12. Claimed Contributions of This Draft

- A Semantic Object Graph Runtime abstraction for language models.
- A separation between low-frequency global relationship construction and high-frequency graph-conditioned execution.
- A structured cache and dependency-version mechanism for incremental recomputation.
- A hierarchical block-diff mechanism designed to align semantic sparsity with GPU-friendly contiguous computation.
- A conditional complexity model and experimental protocol that can falsify the proposed efficiency advantage.

References

- [1] Vaswani, A. et al. Attention Is All You Need. NeurIPS, 2017.
- [2] Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. ICML/PMLR 119, 5156–5165, 2020.
- [3] Schlag, I., Irie, K., Schmidhuber, J. Linear Transformers Are Secretly Fast Weight Programmers. ICML/PMLR 139, 9355–9366, 2021.
- [4] Kitaev, N., Kaiser, Ł., Levskaya, A. Reformer: The Efficient Transformer. ICLR, 2020.
- [5] Gu, A., Dao, T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752, 2023.
- [6] Kimi Team et al. Kimi Linear: An Expressive, Efficient Attention Architecture. arXiv:2510.26692, 2025.
- [7] Veličković, P. et al. Graph Attention Networks. ICLR, 2018.
- [8] Kim, Y., Denton, C., Hoang, L., Rush, A. M. Structured Attention Networks. ICLR, 2017.
- [9] Loynd, R. et al. Working Memory Graphs. ICML/PMLR 119, 6404–6414, 2020.
- [10] Dao, T. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS, 2022.
- [11] Dao, T. et al. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. ICLR, 2024.
- [12] Beltagy, I., Peters, M. E., Cohan, A. Longformer: The Long-Document Transformer. arXiv:2004.05150, 2020.
- [13] Child, R. et al. Generating Long Sequences with Sparse Transformers. arXiv:1904.10509, 2019.
- [14] Liu, Y. et al. Blockwise Parallel Transformer for Long Context Modeling. Related line of chunk/block efficient sequence modeling; see contemporary long-context literature.
- [15] McKinley, K. S. et al. Compiler and incremental computation literature on dependency-directed recomputation; the analogy motivates the runtime design but is not claimed as a new compiler technique.

Originality and attribution note. This manuscript presents the SOGR architecture as a research proposal derived from the author's own conceptual development. It intentionally distinguishes the proposed combination and system-level abstraction from prior work on linear attention, sparse attention, graphs, caches, and hybrid architectures. The paper does not claim that every individual component is novel, nor does it claim proven priority without a comprehensive prior-art search. All numerical complexity statements are analytical conditions or illustrative examples unless explicitly identified as reported results from cited work.